



Parul University

FACULTY OF ENGINEERING & TECHNOLOGY

BACHELOR OF TECHNOLOGY

Enterprise Programming using Java (303105310)

5th SEMESTER

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
ENGINEERING DEPARTMENT

Laboratory Manual

CERTIFICATE

This is to certify that

Mr. _____ Ansari Mohammed Junaid _____ with enrollment
number _____ 2403031460571 _____ has successfully completed his/her laboratory experiments
in the - **Enterprise Programming using Java Laboratory (303105310)** from the department of
Artificial Intelligence & Machine Learning during the academic year 2026-27.



Date of Submission:.....

Staff In charge:.....

Head of Department:.....

TABLE OF CONTENT

Sr. No.	Experiment Title	Page No.		Date of Performance	Date of Submission	Marks	Sign
		To	From				
1.	Practical-1						
	Write a program to insert and retrieve the data from database using JDBC.						
2.	Practical-2						
	Write a program to demonstrate the use of Prepared Statement and Result Set interface.						
3	Practical-3						
	Servlet Programming Servlet Execution on tomcat A servlet program to print hello world A servlet program to display request details A servlet program to handle user form A servlet program to create a cookie A servlet program to display cookie A servlet program to do session tracking Write a program to implement chat Server using Server Socket and Socket class. Write a Servlet program to send username and password using HTML forms and authenticate the user						
4.	Practical - 4						
	JSP Programming JSP program to display hello world. JSP program to demonstrate arithmetic operations JSP program to demonstrate jsp: forward action tag JSP program to request implicit object Developing a web application to insert record into Oracle Database using JSP and JDBC						

Enrollment No.: 2403031460571

5.	Practical-5						
	Create application to store the data in database to perform Hibernate CRUD operations.						
6.	Practical-6						
	Create a application store the data in database to perform Spring CRUD operations.						
7.	Practical - 7						
	Create a web application to store the data in database with spring boot.						



Practical 1: Database Connectivity using JDBC

Aim: To connect a Java application with a MySQL database and perform INSERT and SELECT operations using JDBC.

Objectives

- Understand JDBC components and architecture.
- Establish a database connection using DriverManager.
- Insert a record and retrieve records using Statement.
- Close JDBC resources safely.

Theory

JDBC is a standard Java API for communicating with relational databases. A JDBC program normally loads a driver, creates a Connection, creates a Statement or PreparedStatement, executes SQL, processes the ResultSet, and closes all resources. The Connection represents the database session, Statement executes SQL commands, and ResultSet stores rows returned by a SELECT query.

Software Requirements

- JDK 8 or above
- Eclipse or IntelliJ IDEA
- MySQL Server
- MySQL Connector/J driver

Procedure / Algorithm

1. Create a database named studentdb and a table named student.
2. Add the MySQL Connector/J JAR to the project classpath.
3. Create a Java class named JDBCInsertRetrieve.
4. Load the JDBC driver and establish a connection.
5. Execute an INSERT query and then a SELECT query.
6. Display the retrieved rows and close resources.

Program Code

SQL Script



```
CREATE DATABASE IF NOT EXISTS studentdb;  
USE studentdb;  
CREATE TABLE IF NOT EXISTS student(  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    course VARCHAR(50)  
);
```

JDBCInsertRetrieve.java

```
import java.sql.*;  
  
public class JDBCInsertRetrieve {  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/studentdb";  
        String user = "root";  
        String password = "root";  
  
        try {  
            Class.forName("com.mysql.cj.jdbc.Driver");  
            Connection con = DriverManager.getConnection(url, user, password);  
            Statement st = con.createStatement();  
  
            st.executeUpdate(  
                "INSERT INTO student VALUES (1, 'Aarav Shah', 'AIML')");  
  
            ResultSet rs = st.executeQuery("SELECT * FROM student");  
  
            while (rs.next()) {  
                System.out.println(rs.getInt("id") + " "  
                    + rs.getString("name") + " "  
                    + rs.getString("course"));  
            }  
  
            rs.close();  
            st.close();  
            con.close();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Output

```
1 Aarav Shah AIML
```

Result

The Java application connected successfully to MySQL and performed insertion and retrieval through JDBC.



Viva Questions

1. What is JDBC?
2. What is the role of DriverManager?
3. Differentiate executeQuery() and executeUpdate().
4. What is a ResultSet?
5. Why should JDBC resources be closed?

Conclusion

Thus, database connectivity and basic INSERT and SELECT operations were successfully implemented using JDBC.



Practical 2: PreparedStatement and ResultSet

Aim: To perform parameterized database operations using PreparedStatement and retrieve records using ResultSet.

Objectives

- Understand parameterized SQL statements.
- Use setter methods to bind values.
- Reduce SQL injection risk.
- Retrieve and display employee records.

Theory

PreparedStatement is a precompiled SQL statement containing placeholders represented by ?. Values are supplied through methods such as setInt(), setString(), and setDouble(). It is safer than concatenating user input into SQL statements. ResultSet represents the rows returned by a SELECT query and provides methods to move through and read those rows.

Software Requirements

- JDK 8 or above
- Eclipse IDE
- MySQL Server
- MySQL Connector/J driver

Procedure / Algorithm

1. Create the employee table.
2. Create a connection to studentdb.
3. Prepare an INSERT statement with three placeholders.
4. Bind values and execute the statement for two employees.
5. Prepare and execute a SELECT statement.
6. Traverse the ResultSet and display the records.

Program Code

SQL Script



```
USE studentdb;  
CREATE TABLE IF NOT EXISTS employee(  
    empid INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary DOUBLE  
);
```

PreparedStatementDemo.java

```
import java.sql.*;  
  
public class PreparedStatementDemo {  
    public static void main(String[] args) throws Exception {  
        Connection con = DriverManager.getConnection(  
            "jdbc:mysql://localhost:3306/studentdb", "root", "root");  
  
        PreparedStatement ps = con.prepareStatement(  
            "INSERT INTO employee VALUES (?, ?, ?)");  
  
        ps.setInt(1, 101);  
        ps.setString(2, "Riya Mehta");  
        ps.setDouble(3, 45000);  
        ps.executeUpdate();  
  
        ps.setInt(1, 102);  
        ps.setString(2, "Karan Patel");  
        ps.setDouble(3, 52000);  
        ps.executeUpdate();  
  
        PreparedStatement select = con.prepareStatement(  
            "SELECT * FROM employee");  
        ResultSet rs = select.executeQuery();  
  
        while (rs.next()) {  
            System.out.println(rs.getInt("empid") + " "  
                + rs.getString("name") + " "  
                + rs.getDouble("salary"));  
        }  
  
        rs.close();  
        select.close();  
        ps.close();  
        con.close();  
    }  
}
```

Output

```
101 Riya Mehta 45000.0  
102 Karan Patel 52000.0
```

Result



Parameterized INSERT and SELECT operations were successfully performed using PreparedStatement and ResultSet.

Viva Questions

1. Why is PreparedStatement preferred over Statement?
2. How does it help prevent SQL injection?
3. What is the use of setString()?
4. What is the default ResultSet traversal direction?
5. What is the difference between executeQuery() and executeUpdate()?

Conclusion

Thus, secure parameterized database operations and record retrieval were successfully implemented.



Practical 3: Servlet Programming

Aim: To develop a dynamic web application using Java Servlets and Apache Tomcat.

Objectives

- Understand the Servlet lifecycle.
- Configure a Dynamic Web Project.
- Process GET and POST requests.
- Read form parameters and display a response.

Theory

A Servlet is a server-side Java class used to process client requests and generate dynamic responses. Its lifecycle is controlled by the servlet container and includes `init()`, `service()`, and `destroy()`. `HttpServlet` provides `doGet()` and `doPost()` methods. `HttpServletRequest` supplies request information, while `HttpServletResponse` is used to send output to the browser.

Software Requirements

- JDK 8 or above
- Eclipse IDE for Enterprise Java
- Apache Tomcat 9
- Web browser

Procedure / Algorithm

1. Create a Dynamic Web Project named ServletDemo.
2. Configure Apache Tomcat as the target runtime.
3. Create a servlet mapped to /hello.
4. Create a servlet mapped to /processForm.
5. Create an HTML form that submits name and email using POST.
6. Run the project on Tomcat and test both URLs.

Program Code

HelloServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req,
                        HttpServletResponse res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h2>Hello World from Servlet!</h2>");
    }
}
```

form.html

```
<html>
<body>
<form action="processForm" method="post">
    Name: <input type="text" name="uname"><br>
    Email: <input type="text" name="email"><br>
    <input type="submit" value="Submit">
</form>
</body>
</html>
```

FormServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/processForm")
public class FormServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req,
                        HttpServletResponse res)
        throws ServletException, IOException {
        String name = req.getParameter("uname");
        String email = req.getParameter("email");
        res.setContentType("text/html");
        res.getWriter().println("Welcome, " + name + " (" + email + ")");
    }
}
```

Output

```
Hello World from Servlet!
Welcome, Aarav Shah (aarav@example.com)
```



Result

The Servlet project was deployed on Tomcat and successfully processed browser requests and form data.

Viva Questions

1. What is a Servlet?
2. Explain init(), service(), and destroy().
3. Differentiate doGet() and doPost().
4. What is @WebServlet?
5. What is the role of Tomcat?

Conclusion

Thus, basic Servlet programming and HTML form processing were successfully implemented.

Practical 4: JSP Programming

Aim: To create JSP pages, use JSP implicit objects and action tags, and insert a record into a database using JSP and JDBC.

Objectives

- Understand JSP translation and execution.
- Use JSP expressions and scriptlets.
- Demonstrate arithmetic operations.
- Use the jsp:forward action tag.
- Insert a record into MySQL using JDBC.

Theory

JavaServer Pages (JSP) is a server-side technology used to create dynamic web pages. A JSP page is translated into a servlet by the container. JSP provides implicit objects such as request, response, out, session, application, and exception. Action tags such as jsp:forward allow a page to transfer control to another resource. JSP can communicate with a database through JDBC, although separating Java logic into servlets or service classes is preferable in larger applications.

Software Requirements

- JDK 8 or above
- Eclipse IDE for Enterprise Java
- Apache Tomcat 9
- MySQL Server
- MySQL Connector/J driver

Procedure / Algorithm

1. Create a Dynamic Web Project named JSPDemo.
2. Configure Tomcat and add the JDBC driver.
3. Create a JSP page for arithmetic operations.
4. Create a second JSP page and forward to it.
5. Create a JSP page containing a form for student data.
6. Use JDBC to insert the submitted values into MySQL.
7. Run the pages through the browser.

Program Code



arithmetic.jsp

```
<html><body>
<h2>JSP Arithmetic Operations</h2>
<%
    int a = 20;
    int b = 5;
%>
Addition: <%= a + b %><br>
Subtraction: <%= a - b %><br>
Multiplication: <%= a * b %><br>
Division: <%= a / b %><br>
</body></html>
```

forward.jsp

```
<html><body>
<h2>Forwarding page...</h2>
<jsp:forward page="welcome.jsp"/>
</body></html>
```

welcome.jsp

```
<html><body>
<h2>Welcome to JSP Programming</h2>
</body></html>
```

insert.jsp

```
<%@ page import="java.sql.*" %>
<html><body>
<%
    String name = request.getParameter("name");
    String course = request.getParameter("course");

    if (name != null && course != null) {
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb", "root", "root");
        PreparedStatement ps = con.prepareStatement(
            "INSERT INTO student(id,name,course) VALUES(?,?,?)");
        ps.setInt(1, 10);
        ps.setString(2, name);
        ps.setString(3, course);
        ps.executeUpdate();
        ps.close();
        con.close();
        out.println("Record inserted successfully.");
    }
%>
<form method="post">
    Name: <input name="name"><br>
    Course: <input name="course"><br>
    <input type="submit" value="Insert">
</form>
</body></html>
```

Output

```
JSP Arithmetic Operations
Addition: 25
Subtraction: 15
Multiplication: 100
Division: 4
Record inserted successfully.
```

Result

JSP pages were executed successfully, arithmetic operations and forwarding were demonstrated, and a record was inserted into the database using JSP and JDBC.

Viva Questions

1. What is JSP?
2. What are JSP implicit objects?
3. What is the use of jsp:forward?
4. How is JSP translated by the container?
5. Why is MVC preferred over putting JDBC code directly in JSP?



Conclusion

Thus, JSP pages, implicit objects, action tags, arithmetic operations, and database insertion were successfully demonstrated.

Practical 5: Hibernate CRUD Operations

Aim: To create a Hibernate application that performs Create, Read, Update, and Delete operations on a database table.

Objectives

- Understand ORM and Hibernate.
- Configure a Hibernate SessionFactory.
- Map a Java class to a database table.
- Perform CRUD operations using Hibernate APIs.

Theory

Hibernate is an Object Relational Mapping framework that maps Java objects to relational database tables. It reduces the need for handwritten SQL and provides objects such as SessionFactory, Session, Transaction, and Query. An entity class is mapped using annotations such as @Entity, @Table, and @Id. CRUD operations are performed inside transactions.

Software Requirements

- JDK 8 or above
- Eclipse or IntelliJ IDEA
- Maven
- MySQL Server
- Hibernate ORM dependencies

Procedure / Algorithm

1. Create a Maven project and add Hibernate and MySQL dependencies.
2. Create a Student entity class with id, name, and course.
3. Configure Hibernate using hibernate.cfg.xml.
4. Build a SessionFactory.
5. Open a Session and begin a transaction.
6. Save, retrieve, update, and delete a Student object.
7. Commit the transaction and close resources.

Program Code

Student.java



```
import javax.persistence.*;

@Entity
@Table(name="student")
public class Student {
    @Id
    private int id;
    private String name;
    private String course;

    public Student() {}

    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getCourse() { return course; }
    public void setCourse(String course) { this.course = course; }
}
```

HibernateCRUD.java

```
import org.hibernate.*;
import org.hibernate.cfg.Configuration;

public class HibernateCRUD {
    public static void main(String[] args) {
        SessionFactory factory = new Configuration()
            .configure().addAnnotatedClass(Student.class)
            .buildSessionFactory();

        Session session = factory.openSession();
        Transaction tx = session.beginTransaction();

        Student s = new Student(1, "Aarav", "AIML");
        session.save(s);

        Student found = session.get(Student.class, 1);
        found.setCourse("Computer Engineering");
        session.update(found);

        Student deleted = session.get(Student.class, 1);
        session.delete(deleted);

        tx.commit();
        session.close();
        factory.close();
        System.out.println("Hibernate CRUD completed.");
    }
}
```

Output

```
Hibernate CRUD completed.
```

Result

The entity was mapped successfully and Hibernate CRUD operations were executed within a transaction.

Viva Questions

1. What is ORM?
2. What is the role of SessionFactory?
3. What is an entity?
4. Why are transactions required?
5. Differentiate save(), update(), and delete().

Conclusion

Thus, Create, Read, Update, and Delete operations were successfully implemented using Hibernate.

Practical 6: Spring CRUD Operations

Aim: To create a Spring application that performs CRUD operations using dependency injection and a data access layer.

Objectives

- Understand the Spring IoC container.
- Use dependency injection.
- Create a repository and service layer.
- Perform CRUD operations using Spring JDBC.

Theory

Spring is a Java framework that provides Inversion of Control and Dependency Injection. Instead of creating every dependency manually, the Spring container creates and wires objects called beans. A typical application is divided into controller, service, and repository layers. Spring JDBC provides JdbcTemplate, which simplifies SQL execution and resource management.

Software Requirements

- JDK 8 or above
- Maven
- Eclipse or IntelliJ IDEA
- MySQL Server
- Spring Framework and Spring JDBC dependencies

Procedure / Algorithm

1. Create a Maven project and add Spring Context, Spring JDBC, and MySQL dependencies.
2. Create a Student model class.
3. Create a repository using JdbcTemplate.
4. Create a service class and inject the repository.
5. Configure the DataSource and JdbcTemplate bean.
6. Call the service methods to insert, read, update, and delete records.

Program Code

StudentRepository.java

```
import org.springframework.jdbc.core.JdbcTemplate;

public class StudentRepository {
    private JdbcTemplate jdbcTemplate;

    public void setJdbcTemplate(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public int insert(int id, String name, String course) {
        return jdbcTemplate.update(
            "INSERT INTO student VALUES (?, ?, ?)",
            id, name, course);
    }

    public int updateCourse(int id, String course) {
        return jdbcTemplate.update(
            "UPDATE student SET course=? WHERE id=?",
            course, id);
    }

    public int delete(int id) {
        return jdbcTemplate.update(
            "DELETE FROM student WHERE id=?", id);
    }
}
```

SpringCRUD.java

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class SpringCRUD {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("beans.xml");

        StudentRepository repo =
            (StudentRepository) context.getBean("studentRepository");

        repo.insert(5, "Riya", "AIML");
        repo.updateCourse(5, "IT");
        repo.delete(5);

        System.out.println("Spring CRUD completed.");
    }
}
```

Output

```
Spring CRUD completed.
```



Result

Spring successfully created and injected the repository bean, and CRUD database operations were executed through JdbcTemplate.

Viva Questions

1. What is IoC?
2. What is dependency injection?
3. What is a Spring bean?
4. What is JdbcTemplate?
5. Explain the repository and service layers.

Conclusion

Thus, dependency injection and database CRUD operations were successfully implemented using Spring.

Practical 7: Spring Boot Database Web Application

Aim: To develop a Spring Boot web application that stores and displays records in a MySQL database.

Objectives

- Understand Spring Boot auto-configuration.
- Create REST or MVC endpoints.
- Connect Spring Boot to MySQL using Spring Data JPA.
- Create, save, and retrieve entity records.

Theory

Spring Boot simplifies Spring application development by providing auto-configuration, starter dependencies, an embedded server, and production-oriented defaults. Spring Data JPA provides repository interfaces such as JpaRepository, which automatically supplies common CRUD operations. A Spring Boot application can expose web endpoints through controllers and connect to MySQL through application.properties.

Software Requirements

- JDK 8 or above
- Maven
- Spring Tool Suite or IntelliJ IDEA
- MySQL Server
- Spring Boot Web and Spring Data JPA starters

Procedure / Algorithm

1. Create a Spring Boot project with Web, Spring Data JPA, and MySQL Driver dependencies.
2. Create the Student entity.
3. Create StudentRepository extending JpaRepository.
4. Create a controller with POST and GET endpoints.
5. Configure database properties.
6. Run the application and test the endpoints using a browser or Postman.

Program Code

Student.java

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Student {
    @Id
    private int id;
    private String name;
    private String course;

    public Student() {}

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getCourse() { return course; }
    public void setCourse(String course) { this.course = course; }
}
```

StudentRepository.java

```
import org.springframework.data.jpa.repository.JpaRepository;
public interface StudentRepository
    extends JpaRepository<Student, Integer> {
}
```

StudentController.java

```
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {
    private final StudentRepository repository;

    public StudentController(StudentRepository repository) {
        this.repository = repository;
    }

    @GetMapping
    public List<Student> getAll() {
        return repository.findAll();
    }

    @PostMapping
    public Student save(@RequestBody Student student) {
        return repository.save(student);
    }
}
```

application.properties



```
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Output

```
GET /students
[]

POST /students
{"id":1,"name":"Aarav","course":"AIML"}
```

Result

The Spring Boot application started successfully, connected to MySQL, and exposed endpoints for storing and retrieving student records.

Viva Questions

1. What is Spring Boot?
2. What is auto-configuration?
3. What is JpaRepository?
4. What is the purpose of application.properties?
5. Differentiate @Controller and @RestController.

Conclusion

Thus, a Spring Boot web application connected to MySQL and performed database storage and retrieval through REST endpoints.